Light Deploy 项目使用说明书

> 适用对象：公司内需要使用本平台进行项目部署 / 发布的运维同学 文档定位：一份「拿到就能用」的操作手册，不要求先读代码或设计文档；看完即可独立把项目接进平台并发版 版本基线：2026-09-26 ｜ 配套后端已编译通过、前端管理端已构建通过 权威约束：本说明若与 `AGENTS.md`、各 `docs/Xxx模块.md` 冲突，以那两份为准

---

1. 这平台是干什么的

一句话：**把「手工 SSH 部署脚本」变成一个团队都能安全使用的平台。**

你以前怎么发版？登录堡垒机 → SSH 到目标机 → 拉代码 / 传 jar / 改配置 / `systemctl restart` / 看日志 / 出问题再回退。Light Deploy 把这些动作收敛成一个网页：登记服务器、建项目、点「构建」、点「发布」，剩下的克隆、构建、分发、健康检查、日志、回滚都交给平台。

- 平台纳管多台目标 Linux 服务器（基线 6 台起），不同服务器上的项目**可并发**发布；
- 同一台服务器同一时刻默认只跑一个发布任务；
- 支持三种运行时：**DOCKER / JAR / STATIC**；
- 发布过程日志实时滚动，失败按策略停后续批次，**支持一键回滚到任意保留版本**；
- 凭据（Git 令牌 / 私钥 / 镜像仓库密码）全程加密，页面不回显明文。

> 注意：平台**自身**目前没有自动化部署能力，平台本身更新仍需手工（见 §9）。你用它管的是「业务项目」，不是平台自己。

---

2. 必须先理解的核心概念

第一次用最怕「术语对不上操作」。下面 8 个词理解了，后面所有页面都顺了。

概念	一句话解释	在平台里的位置
服务器（server）	一台被平台纳管的 Linux 机器，存 SSH 连接信息。**构建机**和**目标机**都是服务器	服务器管理
凭据（credential）	Git 令牌 / 私钥 / 镜像仓库密码，全局资源，挂在「系统设置」下	系统设置 → 凭据管理
镜像仓库（registry）	Docker 镜像推送目标（Harbor / 阿里云 ACR / Docker Hub）。**只有 DOCKER 运行时才需要**	系统设置 → 镜像仓库
项目（project）	一条「可独立构建、独立发布、独立回滚」的服务单元（一个 jar / 一个	项目管理

概念	一句话解释	在平台里的位置
	容器 / 一个静态站），**不是**「一个 Git 仓库」	
环境（environment）	dev / test / prod 之类的发布环境，决定默认发布策略与批次大小	项目管理 → 环境管理
部署目标（target）	把「项目 × 环境」绑定到具体哪台服务器；支持主目标、发布顺序	项目管理 → 部署目标
发布版本（release）	构建成功后自动生成的一条「可发布」记录，形如 `auth-7888-v3-887b603c`	发布时下拉选择
发布策略（strategy）	单目标 / 全量 / 滚动，决定「一批发几台、失败了停不停」	环境默认，发布时可覆盖

两个最容易混淆的点：

- **构建机 vs 目标机**：构建机是「拉代码、跑编译、产出 jar/镜像」的机器（要装 git、JDK、Maven/Docker）；目标机是「最终跑服务」的机器。两者可以是同一台，但生产建议分开。
- **构建（build）≠ 发布（deploy）**：构建产出「可发布版本」，发布把版本发到目标机。平台是「先构建出版本 → 再选版本发布」两段式，中间用发布版本号衔接。

---

3. 第一次上手：环境准备清单

在网页上点之前，先确认机器侧满足这些条件，能省掉 90% 的「构建暴雷」：

① 构建机（必选，至少要一台） – 安装：git、JDK（按项目要求，如 JDK 21）、Maven 或 Gradle 或 Node（按项目）、DOCKER 运行时（仅 DOCKER 项目需要）； – 在「服务器管理」登记这台机器，并填好三个目录（详见 §4.1）：构建工作区、产物持久目录、部署根目录； – 建议在该服务器档案里配「构建环境初始化脚本」，补 `mvn` / `docker` 等 PATH（非交互 SSH 不加载 `/etc/profile`，见 §7 排查）。

② 目标机（每个要部署的服务的机器） – 安装：对应运行时依赖（JAR 需 systemd + JDK；DOCKER 需 docker；STATIC 需 Nginx）； – **JAR 项目的 SSH 登录用户需要 root**（或能免密 `systemctl`）：平台初始化脚本直接执行 `systemctl daemon-reload/enable`，不带 sudo； – 在「服务器管理」登记，填好「部署根目录」。

③ 代码仓库 & 凭据 – 私有仓库提前在代码平台生成访问令牌 / 部署公钥（见 §4.2）； – DOCKER 项目提前准备好镜像仓库与登录凭据。

> 经验：新环境第一次跑通，直接用本平台仓库自举最省事，完整样例见 §8。

---

4. 分步操作指南（对照菜单）

平台菜单为二级结构，一级是分组目录，页面挂在二级。下面是标准上手顺序。

4.1 系统设置 → 服务器管理（先纳管机器）

1. 进入 **服务器管理 → 服务器列表**，点「新增」：
    - 服务器名称、主机地址、SSH 端口（默认 22）、登录用户、认证类型（密码 / 私钥）、**凭据（密文，必填，只写不读）**；
    - 环境标签（开发 / 测试 / 生产，仅用于列表筛选，不决定发布目标）；
    - 状态：启用。
2. 表单底部三个目录（按机器维护，**建议显式填写**，不要只靠默认值）：
    - **构建工作区 `build_workspace_dir`**：构建时 git clone、跑命令的临时目录，每次构建先清空再重建；
    - **产物持久目录 `build_artifact_dir`**：构建成功后 jar / 静态产物转存处，供发布下载；**绝不能放在工作区里**（否则构建清理会把已发布产物一起删掉）；
    - **部署根目录 `agent_base_dir`**：目标机上应用落盘根，JAR 未配 `work_dir` 时落在 `{部署根目录}/apps/{项目编码}`（固定目录，发版原地替换 jar，不建版本子目录）。**项目运行配置里填了 `work_dir` 后就完全改到那个目录**，这里的部署根目录对该项目不再生效；STATIC 默认不用它（走 `/usr/share/nginx/html/{项目编码}`）。
    - 单机自测可填：`/lx/light-deploy/workspace`、`/lx/light-deploy/artifacts`、`/lx/light-deploy/agent`，互不包含即可。
3. （构建机专属）若 `mvn` / `docker` 不在默认 PATH，填写「构建环境初始化脚本」，如：

```
export PATH=/opt/apache-maven-3.8.8/bin:$PATH
```

4. 列表行「连通性测试」按钮：提交后弹日志抽屉实时看结果，确认 SSH 通道、运行时、磁盘、部署目录都正常。

4.2 系统设置 → 凭据管理（Git / 镜像仓库登录用）

凭据是全局资源，登记一次，多个项目可共用。四种类型，**只填前三种**：

类型	明文内容格式	示例	适用仓库
GIT_TOKEN	用户名:令牌，或只填令牌	wangan2022:a1b2c3...	HTTPS 形式仓库（https://...）
SSH_KEY	**完整 PEM 私钥**（含首尾行、保留换行）	-----BEGIN OPENSSH PRIVATE KEY-----…	SSH 形式仓库（git@主机:组/仓库.git）
REGISTRY	用户名:密码（按第一个冒号切分）	light-deploy:Abc123456	镜像仓库登录

类型	明文内容格式	示例	适用仓库
SSH_PASSWORD	——	——	**无消费方，不要选**（服务器 SSH 密码在服务器档案里维护）

填写要点： – **GIT_TOKEN 是「当密码用的令牌」，不是账号密码**。HTTPS 地址必须 `https://` 开头；令牌填在冒号后那段（只填令牌时平台自动补 `oauth2:` 占位）。 – **SSH_KEY 填私钥不是公钥**：`ssh-keygen` 生成的私钥（无 `.pub` 后缀）完整内容；对应**公钥**上传到代码平台（部署公钥 / Deploy key，默认只读即可）。私钥**不能带口令**。 – 凭据明文只进请求、不回显、不进日志；被项目或仓库引用时删除会被拦截（先解绑再删）。

4.3 系统设置 → 镜像仓库（仅 DOCKER 项目需要）

1. 新增：仓库名称、地址（**不带 `https://` 前缀**，如 `registry.cn-hangzhou.aliyuncs.com`）、命名空间（可选）、登录凭据（选 `REGISTRY` 类型）、状态启用。
2. 公共仓库（如 Docker Hub 匿名）可不绑凭据；私有仓库必须绑。
3. 仓库地址可达性**不预检**，配错要到首次推送才暴露——所以第一次发 Docker 项目务必看构建日志。

4.4 项目管理 → 项目列表（建项目，关键一步）

点「新增」，填：

基本信息 – 项目编码（`code`，`^[A-Za-z0-9_-]{2,64}$`，唯一，后续用于产物目录 / systemd 服务名 / 容器名 / 版本号，**产生过构建发布记录后不可改**）； – 项目名称； – **运行时类型（runtime_type）**：DOCKER / JAR / STATIC —— 见 §5 决策； – 代码来源：GIT（默认，必填 Git 地址 + 分支）；UPLOAD 在 v1 未开放； – Git 地址、分支；Git 凭据（私有仓库选 §4.2 建的 GIT_TOKEN / SSH_KEY）； – **构建模式（build_mode）**：随运行时联动，选项见 §5； – **镜像仓库**：仅 DOCKER 运行时出现，必绑； – **构建机（build_server_id）**：选 §4.1 登记的构建机（选填，留空回落系统默认）。**强烈建议绑定**——绑定后才能做「项目连通性测试」，且构建默认带出该机； – 保留版本数（默认 5，决定可回滚深度）。

三段高级配置（构建 / 运行 / 健康检查） 编辑器支持「表单」和「JSON」双模式，下面给最常填的字段（完整键名见 `docs/项目模块.md` §2.3）：

构建配置 `build_config_json`：

```
{
  "build_command": "mvn -B clean package -DskipTests",
  "artifact_path": "target/auth-app-*.jar",
  "config_paths": "src/main/resources/application.yml"
}
```

- 多模块仓库用 `module_name` + `module_dir` 指定子模块；**填了 `module_dir` 就不要再在命令里写 `-pl`**（会报 reactor 找不到模块）。

运行配置 `runtime_config_json`（按运行时填对应键）： – JAR：`work_dir`（**固定工作目录**：填了就完全覆盖服务器档案的部署根目录，必须写绝对路径；留空才回落 `{部署根目录}/apps/{编码}`）、`service_name`、`jvm_options`、`java_bin`、`spring_profiles_active`、`config_dir`、`config_file`； – DOCKER：`container_name`、`port_mappings`（如 `8080:8080`）、`extra_docker_args`； – STATIC：`static_root`（Nginx root，默认 `/usr/share/nginx/html/{编码}`）； – `deploy_base` 当前版本后端未消费，别填。

健康检查 `health_config_json`（**不配则跳过检查**）：

```
{
  "type": "TCP",
  "port": 2027,
  "retry_count": 20,
  "interval_seconds": 5
}
```

- HTTP 探活路径必须**免鉴权、直接返回 200**（不跟重定向、不返回 302/401/403，否则判不健康）；
- 探测在**平台侧**发起，平台需能访问目标机业务端口。

建完项目后，先做「项目连通性测试」（列表行按钮）：在绑定的构建机上跑 `git ls-remote` + `docker login` + 工具链探测，把「仓库拉不动 / 凭据失效 / 镜像推不上」提前暴露，避免构建时才暴雷。日志实时滚动，看结论行即可。

4.5 项目管理 → 环境管理

1. 新增环境：编码（如 `dev` / `test` / `prod`）、名称；
2. **发布策略（deploy_strategy）** —— 这里配的是**该环境的默认值**，新建部署时可在表单里临时覆盖，覆盖只影响那一次：

策略	行为	适用
`SINGLE_TARGET` 单目标	只发一台：优先主目标，没有主目标取发布顺序最靠前的一台。**是策略层强制只发一台**，绑了 4 台也只发主目标那台	环境本来就是单机部署，或先发一台做验证
`ALL_AT_ONCE` 全量	启用目标全部参与，算一批，**批内逐台串行**（不是并发），某台失败**不中止**，其余继续发完	无状态服务、能接受一起上，失败后有别的兜底
`ROLLING` 滚动	按 `batch_size` 切批，批内逐台串行，**任一批失败立即中止**，后续目标置「已跳过」、保持旧版本不动	想「先发一批看看，炸了就停」，把影响面限制在已发的那几台

两个容易误解的点：**全量 ≠ 并发**（总耗时约等于各台之和）；**滚动中止只保护还没发的机器**，失败那台不会自动还原——它是 FAILED 记录，回滚按钮只对成功记录开放，要收尾就走「新建部署选旧版本、目标只勾那一台」重发。

3. 批次大小 `batch_size`：**只对滚动生效**，单目标与全量不读这个值；表示每批包含几台目标；**必须小于目标总数才有分批效果**（3 台填 3 只剩一批，滚动退化成全量，建议填 1 或 2）。批量大小在新建部署里不能覆盖，只能取环境上配置的值。默认目标服务器（可选）；Alops 开关（v1 仅占位）。

4.6 项目管理 → 部署目标

把「项目 × 环境」绑定到具体服务器： – 项目、环境、服务器三级下拉（服务器仅启用项）； – 可设**主目标**（同项目环境内互斥，最多一个，单目标策略优先发它）； – 发布顺序 `deploy_order`：**数字小的先发**；启用目标先按此值升序，值相同再按绑定创建时间升序。**所有策略都按它展开目标**，不只是滚动；单目标策略下它还有第二重作用——没有主目标时取顺序最靠前的那台作为发布对象； – 目标差异化运行配置（覆盖项目级 `runtime_config_json`，结构一致）； – 状态启用。

> 目标列表在**创建部署时快照**，之后改拓扑不影响历史部署（保证回滚发对机器）。

4.7 构建管理 → 构建列表（产出可发布版本）

进 **项目管理 → 项目列表**，行内「触发构建」即可（或进构建管理页）： – 弹窗可选填**版本序号**：留空由平台自动递增取号（`MAX(seq)+1`）；填写则在创建时即校验项目内未被占用； – 点确定后「创建即执行」——只落记录并返回任务 ID，**真正的克隆/构建在后台跑**； – 自动弹出日志抽屉，实时看 `git clone` → 构建命令 → 产物登记； – 构建成功自动生成一条「可发布」`release_record`（版本号 `{编码}-v{序号}-{提交短码}`），出现在发布版本下拉里。

列表行操作： – **查看日志**：增量拉取实时日志； – **产物**：看本次构建产物（镜像 tag / jar SHA256）； – **重新构建**：针对**同一项目**再触发一次**全新**构建（克隆分支**最新 HEAD**，不重建旧记录那次的包，也不按指定 commit 复现）； – **删除**：仅终态（成功 / 失败）可删。

4.8 发布管理 → 部署记录（发到目标机）

进 **发布管理 → 部署记录**，点「新建部署」： – **项目 → 环境 → 可发布版本** 三级联动（版本下拉只列该项目「可发布」状态的历史版本，历史版本永不置废弃，随时可重发）； – 策略留空即沿用环境默认；这里选的只覆盖本次、不改环境档案（批次大小不能在这里覆盖，只取环境上的值），选完环境后表单会回显「环境默认：xxx」作对照； – 确认后创建即执行，目标按策略编排，逐台执行对应运行时链路（见 §6），再做健康检查。

发布过程中： – **目标抽屉**：看每台机器「当前版本 → 本次版本」对照与健康检查结果； – **发布日志**：实时滚动； – 状态：排队 → 发布中 → 成功 / 失败 / 已取消 / 回滚中 / 已回滚。

列表行按状态显隐操作： – **回滚**：仅「成功」且确实可退时开放（按钮可用性看服务端判定，置灰时 tooltip 给原因）； – **取消**：仅排队 / 发布中； – **删除**：仅终态。

> 行内**没有「重新发布」**：部署记录是一次执行的不可变快照，重发同版本请走「新建部署」手选历史版本。

4.9 任务管理 → 任务列表

所有异步任务（SERVER_TEST / PROJECT_TEST / BUILD / DEPLOY）的日志都在这里。`onActivated` 自动刷新。点「查看日志」同款控制台。

---

5. 怎么选适合自己项目的部署方式（核心决策）

这是新人最该看的一节。先定**运行时**，再定**构建模式**。

5.1 先选运行时

你的服务长什么样	运行时 runtime_type	典型场景
Java 应用（Spring Boot jar / 其它可执行 jar）	**JAR**	后端微服务、内部工具
容器化服务（自带 Dockerfile 或要分层构建）	**DOCKER**	K8s 之外的容器部署、需要镜像仓库分发
前端静态产物（Vue / React 的 dist）	**STATIC**	官网、后台管理前端

5.2 再选构建模式（受运行时约束，前端下拉已联动）

运行时	可选构建模式	说明
DOCKER	`PLATFORM_TEMPLATE`	平台生成 Maven 多阶段 Dockerfile，你不用写 Dockerfile
	`PROJECT_DOCKERFILE`	用项目自带的 Dockerfile
	`LAYERED_DOCKER`	分层构建：`Dockerfile.base` + `Dockerfile.app`，基础镜像推仓库、业务镜像基于它（Alops 项目用这个）
JAR	`SCRIPT_DEPLOY`	平台脚本构建产出 jar（v1 实际开放）
	`ARTIFACT_ONLY`	直接发布已有 jar（**v1 未开放**）
STATIC	`SCRIPT_DEPLOY`	平台脚本构建产出静态目录（v1 实际开放）
	`ARTIFACT_ONLY`	直接发布已有静态产物（**v1 未开放**）

> 组合非法会被后端拦截（如 JAR 选了 PROJECT_DOCKERFILE）。前端下拉已按运行时过滤，正常不会选错。

5.3 一页决策表

我想部署…	运行时	构建模式	还需准备
Spring Boot 后端 jar	JAR	SCRIPT_DEPLOY	Git 凭据、构建机装 JDK+Maven、目标机装 JDK+systemd
一个 Docker 微服务（项目自带 Dockerfile）	DOCKER	PROJECT_DOCKERFILE	镜像仓库 + REGISTRY 凭据、构建机装 Docker
想让平台帮我生成 Dockerfile	DOCKER	PLATFORM_TEMPLATE	同上
大镜像想分层提速	DOCKER	LAYERED_DOCKER	同上，且仓库可推 base 镜像
Vue 前端站点	STATIC	SCRIPT_DEPLOY	Git 凭据、构建机装 Node、目标机装 Nginx
直接发一个现成 jar（不重新构建）	—	—	**v1 暂不支持**，需先走构建产出发布版本

关键提醒：v1 只开放 **GIT 来源 + SCRIPT_DEPLOY / 三种 Docker 构建模式**。`ARTIFACT_ONLY`、`UPLOAD` 来源尚未开放，别在界面上找「上传 jar」入口。

---

6. 三条发布链路在目标机上做了什么

了解底层动作，出问题能判断卡在哪一步。

DOCKER `docker pull` → `docker rm -f`（容器名幂等关键）→ `docker run -d --restart=always` → `docker inspect` 确认容器仍运行。镜像直接从仓库拉，不经过平台中转。

JAR（固定工作目录 + 原地替换） – 首次：平台上传 `jar-init.py` / `jar-update.py` / `jar-fallback.py` 到工作目录 `exec/`，生成 `logs/`、`config/`、systemd 单元（`--spring.config.location` 指向 config），**systemd 单元只生成这一次，之后不重写**（运维改过的启动参数被尊重）； – 每次发布固定时序：备份旧 jar 到 `exec/bak`（备份名内嵌被换下包的版本号）→ 投放新 jar → **复核 SHA256** → 写 `VERSION` → 刷新 `exec/deploy.conf` 的 `version=` → `systemctl restart` → `is-active` 确认； – 版本标识用外部 `VERSION` 文件，**不重写 jar 的 MANIFEST.MF**（避免 fat jar 重打包静默变砖）； – 回滚走同一路径，只是投放 `old_version` 的 jar。

STATIC 构建机把 dist `tar -czf` 打包 → SFTP 上传目标机 → 解压到 Nginx root → `nginx -t && nginx -s reload`（目标机没 nginx 只告警不失败）。

> 分发统一走「构建机 → 平台 → 目标机」中转，目标机不需要持有构建机免密通道。

---

7. 构建 / 发布失败怎么排查

日志是第一现场。点「查看日志」看实时输出，关注失败原因那一行（红色 el-alert）。

构建阶段常见坑 – bash: mvn: command not found：构建机非交互 SSH 不加载 profile。在服务器档案配「构建环境初始化脚本」补 PATH，或在 build_command 写绝对路径。 – fatal: could not read Password ... No such device or address：Git 凭据形态与仓库协议不匹配。GIT_TOKEN 配在 https:// 仓库、SSH_KEY 配在 git@ 仓库；令牌必须落在密码位（平台已自动补 oauth2:）。 – 分支不存在：git ls-remote 成功但输出为空，平台已判空报错——核对分支名。 – 构建超时：调大 build_config_json.command_timeout_seconds（默认 1800 秒）。 – Maven 报 Could not find the selected project in the reactor：module_dir 与 -pl 重复指定，二选一。 – JDK 版本错配：日志里 构建工具链版本： 会打出 Maven 用的 JVM，对照报错定位。

发布阶段常见坑 – 健康检查不通过（HTTP 返回 302/401/403、端口不通）：探活路径必须免鉴权直返 200；平台侧需能访问目标机业务端口；先清空 health_config_json 跳过检查验证链路。 – DOCKER 推送失败：镜像仓库地址 / 凭据错，或命名空间无推送权限（后者要真推送才暴露）。 – 目标机 exec/bak 找不到版本：那是**手工应急**用的，平台回滚**不读**它，而是重新发布 old_version 的发布版本（依赖 release_record + 构建机产物）。

---

8. 完整样例：用平台仓库自举（第一次跑通照抄）

本仓库是公开单模块 Maven 项目，产物是可执行 jar，最适合当「第一次验证」样例。

1. 服务器管理：登记一台装好 git/JDK21/Maven 的机器（作构建机，也作目标机），三个目录显式填好；
2. 凭据管理：公开仓库可跳过 Git 凭据；
3. 项目管理 → 新增：
    - 编码 light-deploy，Git 地址 https://gitee.com/wangan2022/light-deploy.git，分支 main；
    - 运行时 **JAR**，构建模式 **SCRIPT_DEPLOY**，镜像仓库留空；
    - 构建配置：{"build_command":"mvn -B clean package -DskipTests","artifact_path":"target/light-deploy-0.0.1-SNAPSHOT.jar","config_paths":"src/main/resources/application.yaml"}；
    - 运行配置：{"service_name":"light-deploy","java_bin":"/usr/bin/java","jvm_options":"-Xms256m -Xmx512m","spring_profiles_active":"prod","config_dir":"config","config_file":"applica（work_dir 留空回落 /apps/light-deploy）；
    - 健康检查先用 TCP：{"type":"TCP","port":2027,"retry_count":20,"interval_seconds":5}（只想验证链路可先清空跳过）；
    - 绑定构建机；
4. 点「项目连通性测试」确认通过；

5. 点「触发构建」，看日志到「构建成功、生成发布版本」；
6. 环境管理：建 `prod` 环境，策略 `SINGLE_TARGET`；
7. 部署目标：把 `light-deploy` × `prod` 绑到那台服务器，设为主目标；
8. 发布管理 → 新建部署：项目 `light-deploy` → 环境 `prod` → 选刚生成的版本 → 确认；
9. 看目标抽屉与日志，状态变「成功」即完成。之后每次迭代：改代码 → 触发构建（新版本号）→ 新建部署选新版本；出问题在部署记录点「回滚」。

---

9. 回滚与版本识别

平台回滚（推荐） – 在「部署记录」行点「回滚」：先弹**预览**（逐台「当前版本 → 将回滚到的版本」、可回退/跳过原因），确认后提交； – 回滚判定唯一入口：状态成功 ∧ 变更过的目标有 `old_version` ∧ 该历史版本的记录与产物仍在（DOCKER 看 image_tag；JAR/STATIC 看 build_server_id + artifact_path）； – 不满足时按钮置灰并给原因（如「首次发布无可回滚」「历史版本产物已不存在」），**不会静默成功**； – 回滚日志独立交代六要素（开始/当前版本→回滚到/起止时间/成功失败/服务是否启动成功）； – 同一条记录只能回滚一次（回滚后变「已回滚」）；要再退一层，找更早那条仍是「成功」的部署记录重发旧版本。

手工应急（平台宕机时） – JAR 目标机 `exec/` 下有 `jar-fallback.py`：`--steps 1` 退一步；备份名形如 `auth-tool_bak_20260925222851_auth-7888-v6-c9bb25d3.jar`，**版本号已内嵌在文件名里**，`ls` 一眼可读；需要严肃确认时用 `sha256sum` 比对 `release_record.sha256`（版本段取自磁盘 `VERSION`，不取库快照）。 – DOCKER 最直接：本机残留旧镜像 `docker run` 即可，不依赖备份。 – **手工换包后平台状态会脱节**：下次发布算 old_version 会错，记得在平台补一次「新建部署、重发对应版本」对齐。

---

10. 注意事项与已知边界

- **平台自身更新需手工**：仓库无 Dockerfile / 部署脚本，平台是「后端 jar（systemd，:2027，context-path `/deploy`）+ 前端 dist（Nginx 托管反代 `/deploy`）」两个交付物；MySQL 结构升级**必须手工执行** `src/main/resources/db/` 下的升级 SQL，JAR 部署链路不执行 DDL。
- **凭据与镜像仓库无「测试连接」按钮**：正确性要到实际拉取 / 推送才暴露。
- **构建机单点**：v1 由一台（或系统默认）构建机承担全部构建，未做集群。
- **产物不可按 commit 复现**：「重新构建」永远克隆分支最新 HEAD，不能按历史 commit 重建；要复现旧代码走部署侧回滚 / 重发旧版本。
- **无「一键重新发布」入口**：重发同版本用「新建部署」手选历史版本（历史版本永不废弃）。
- **删除顺序**：先删部署目标 → 再删服务器 / 项目 / 环境；项目、环境、凭据、仓库删除有引用保护，但服务器删除不反向拦截，注意别留下指向已删服务器的脏目标。
- **回滚中不允许取消**：中途取消会留新旧混合状态。
- **备份无回收策略**：JAR 每次发版在目标机 `exec/bak` 追加备份；构建机产物持久目录随构建增长，均需后续配保留策略。

---

11. 快速排错速查表

现象	大概率原因	怎么办
新增服务器后连通性测试报「已停用 / 有未完成任务」	服务器停用或上次测试没结束	启用 / 等任务终态
构建报 `mvn: command not found`	非交互 SSH 没加载 profile	服务器档案配构建环境初始化脚本补 PATH
私有仓库克隆报 `could not read Password`	凭据形态/协议不匹配	GIT_TOKEN⍰https、SSH_KEY⍰git@；令牌落密码位
构建报 reactor 找不到模块	`module_dir` 与 `-pl` 重复	二选一
发布成功但状态失败	健康检查不通过	探活路径免鉴权直返 200；或先清空 health_config 跳过
回滚按钮置灰	首次发布 / 历史产物已不存在	看 tooltip 原因；走新建部署重发旧版本
部署记录里没有「重新发布」	设计如此（快照不可重跑）	新建部署手选历史版本
列表不显示刚建的构建/部署	任务列表需切回/刷新自动拉	任务、构建、部署列表 `onActivated` 自动刷新

> 文档到此。更多字段口径、状态编码、接口清单见 `docs/` 下各 `Xxx模块.md`；开发约定与硬性约束见根目录 `AGENTS.md`。